

STACK

Q9) Implement Stack using Single Queue

DS	Session	Stack	Question Information	Level 1	Challenge 49
Question description Rathik organized technical round interview in Macrosoft for the set of computer science candidates. The problem is to perform Implement a stack using single queue. you have to use queue data structure, the task is to implement stack using only given queue data structure. Rathik have given the deadline of only 5 minutes to complete the problem. Can you Help the candidates to complete the problem within the specified time limit ? Function Description x is the element to be pushed and s is stack push(s, x) 1) Let size of q be s. 2) Enqueue x to q 3) One by one Dequeue s items from queue and enqueue them. Removes an item from stack pop(s)					

1) Dequeue an item from q

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into stack and m is the number of pop operation need to be performed

next line indicates the n number stack elements

Output Format:

First line indicates top of the element of the stack

second line indicates the top of the element after the pop operation

CODE:

```
C main.c > dequeue
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int data[1005];
6     int front;
7     int rear;
8     int size;
9 } Queue;
10
11 Queue* createQueue() {
12     Queue *q = (Queue *)malloc(sizeof(Queue));
13     q->front = 0;
14     q->rear = -1;
15     q->size = 0;
16     return q;
17 }
18
19 void enqueue(Queue *q, int x) {
20     q->rear++;
21     q->data[q->rear] = x;
22     q->size++;
23 }
24
25 int dequeue(Queue *q) {
26     int val = q->data[q->front];
27     q->front++;
28     q->size--;
29     return val;
30 }
31
32 typedef struct {
33     Queue *q;
34 } Stack;
35
36 Stack* createStack() {
37     Stack *s = (Stack *)malloc(sizeof(Stack));
38     s->q = createQueue();
39     return s;
40 }
41
42 void push(Stack *s, int x) {
43     int sz = s->q->size;
44     enqueue(q: s->q, x);
45     for (int i = 0; i < sz; i++) {
46         enqueue(q: s->q, x: dequeue(q: s->q));
47     }
48 }
49
50 int pop(Stack *s) {
51     if (s->q->size == 0) return -1;
52     return dequeue(q: s->q);
53 }
54
55 int top(Stack *s) {
56     if (s->q->size == 0) return -1;
57     return s->q->data[s->q->front];
58 }
59
60 int main(void) {
61     int n, m;
62     if (scanf(format: "%d %d", &n, &m) != 2) return 0;
63
64     Stack *s = createStack();
65
66     for (int i = 0; i < n; i++) {
67         int val;
68         scanf(format: "%d", &val);
69         push(s, x: val);
70     }
71
72     printf(format: "%d\n", top(s));
73
74     for (int i = 0; i < m; i++) {
75         pop(s);
76     }
77
78     printf(format: "%d\n", top(s));
79
80     return 0;
81 }
```

OUTPUT:

```
[suraj@suraj-83gs test]$ ./main
5 2
1 2 3 4 5
5
3
[suraj@suraj-83gs test]$
```